

M02 - FUNDAMENTOS

Fundamentos da Linguagem

Identificadores - Tipos de Dados - Declarações - Escopo e Visibilidade

Agenda da Aula

01

Identificadores

Regras de nomenclatura e palavras reservadas

02

Literais

Numérico, Caracter, String e Booleano

03

Comentários e Fim de Linha

Estilos -- e /* */ no SQL*Plus

04

Tipos Escalares

BINARY_INTEGER, NUMBER, CHAR, VARCHAR2, DATE, BOOLEAN

05

Atributo %TYPE

Ancoragem ao tipo de coluna ou variável

06

Escopo e Visibilidade

Blocos aninhados, shadowing e labels

07

Declarações e Constantes, NOT NULL e DEFAULT

Sintaxe completa e regras de inicialização

08

Exercício Prático

A02_08 | SELECT INTO com %TYPE

Identificadores

Devem começar com uma letra, seguida de letras, números ou os símbolos \$, _ e #. Têm até 30 caracteres e não diferenciam maiúsculas de minúsculas.

Quando envolvidos por aspas duplas, aceitam caracteres especiais e passam a diferenciar maiúsculas de minúsculas — mantendo o limite de 30 caracteres.

Exemplos válidos: `v_nome`, `c_pi`, `v_sal_bruto`, `"Salário Bruto"`.

Inválidos: `lnome`, `v nome`, `END`, `v-nome`. Veja no script A02_01.

Palavras Reservadas

Palavras com significado especial em PL/SQL não podem ser usadas como nomes de variáveis. Exemplos: BEGIN, END, DECLARE, IF, LOOP, NUMBER, VARCHAR2.

Erro: Palavra Reservada como Variável

```
-- ERRADO: palavra reservada como variável
DECLARE
  NUMBER NUMBER := 10;
BEGIN
  DBMS_OUTPUT.PUT_LINE(NUMBER);
END; -- ORA-06550 / PLS-00320

-- CORRETO: renomeie a variável
DECLARE
  v_number NUMBER := 10; -- OK
BEGIN
  DBMS_OUTPUT.PUT_LINE(v_number);
END;
/
```

Execute em aula: [A02_01_declaracao_basica.sql](#)

BEGIN, END, DECLARE, NUMBER, IF, LOOP, VARCHAR2. Lista em V\$RESERVED_WORDS.

Erro PLS-00103

PLS-00103 em tempo de compilacao. Identificado pelo compilador antes da execucao.

Aspas duplas NÃO resolvem em variáveis

Servem para objetos do schema, não para variáveis locais. Geram ORA-06550 / PLS-00320.

Boa prática

Use prefixos descritivos: v_numero, c_pi, p_nome. Evite conflitos sem precisar de aspas.

Literais: Número, Caracter, String e Booleano

Valor fixo escrito diretamente no código. PL/SQL tem quatro tipos:

- NÚMERO: 100, -125, 12.5, 12E3.
- CARACTER: 'A'. STRING: 'Oracle'.
- BOOLEANO: TRUE, FALSE, NULL.

NUMERO

Inteiros: 100, -125.
Reais: 12.5, .78.
Notação científica: 12E3 = 12000

CARACTER e STRING

Um caractere: 'A' (entre apóstrofes). String: 'PL/SQL'. String vazia: ''. Case-sensitive: 'a' diferente de 'A'. Literal string: 'O"Brien' (apóstrofo duplo).

BOOLEANO

TRUE, FALSE e NULL.
Exclusivo do PL/SQL, não existe em SQL. NULL booleano não é FALSE. Use IS NULL para verificar.

Comentários e Fim de Linha

PL/SQL suporta dois estilos de comentário e dois marcadores de fim: ponto-e-virgula encerra cada comando, barra (/) executa o bloco no SQL*Plus.

```
DECLARE
v_nome VARCHAR2(50); -- comentário de linha
v_sal  NUMBER(10,2); -- ignorado pelo compilador

/* Comentário de bloco.
   Pode ter múltiplas linhas.
   Não pode ser aninhado. */

BEGIN
v_nome := 'Oracle'; -- ; encerra o comando
v_sal  := 4500;
DBMS_OUTPUT.PUT_LINE(v_nome);
END;
/ -- barra encerra e executa
```

-- Comentário de Linha

Dois hifens tornam o restante da linha um comentário. O compilador ignora tudo até o fim da linha.

/* */ Comentário de Bloco

Marca uma região inteira. Pode ter múltiplas linhas. Não pode ser aninhado. Útil para desativar blocos de código.

; Ponto-e-Virgula

Encerra cada COMANDO dentro do bloco. Não encerra o bloco inteiro. Obrigatório em cada instrução.

/ Barra Executora (SQL*Plus)

Em linha separada após END;. Encerra E executa o bloco no SQL*Plus. Sem a barra o cursor fica aguardando mais entrada.

Tipos Escalares: BINARY_INTEGER e NUMBER

BINARY_INTEGER

Inteiros de $-2^{31}+1$ a $2^{31}-1$. Mais rápido que NUMBER para contadores de loop. Subtipos úteis: NATURAL (aceita apenas ≥ 0) e POSITIVE (aceita apenas > 0).

PLS_INTEGER

Usa aritmética de máquina (32 bits). Mais rápido que BINARY_INTEGER em loops intensos. Recomendado pela Oracle para contadores de alta performance a partir do Oracle 10g.

Dica de produção

Declare sempre NUMBER(p,s) para variáveis que vão para colunas de banco.

Use PLS_INTEGER para contadores em loops.

Execute: A02_02_tipos_escalares.sql

NUMBER(precisão, escala)

Tipo numérico padrão Oracle. NUMBER(10,2): 10 dígitos no total, 2 decimais. NUMBER sem parâmetros aceita qualquer valor. Em produção sempre declare precisão e escala.

BOOLEAN

Aceita três valores: TRUE, FALSE e NULL. Exclusivo do PL/SQL. Não existe como tipo de coluna em SQL. NULL booleano não é FALSE. Use IS NULL para verificar.

Tipos: CHAR, VARCHAR2, DATE e BOOLEAN

Tipos de texto de tamanho fixo, variável e tipo data com hora

Demonstração: A02_02_tipos_escalares.sql

```
DECLARE
  v_fixo CHAR(10) := 'ORA';
  v_var VARCHAR2(30) := 'Oracle 19c';
  v_data DATE := SYSDATE;
BEGIN
  DBMS_OUTPUT.PUT_LINE(LENGTH(v_fixo)); -- 10
  DBMS_OUTPUT.PUT_LINE(LENGTH(v_var)); -- 10
  DBMS_OUTPUT.PUT_LINE(
    TO_CHAR(v_data,'DD/MM/YYYY'));
END;
/
```

Execute em aula: A02_02_tipos_escalares.sql

CHAR(10)

Tamanho fixo. LENGTH sempre = 10.
Preenche com espaços à direita.

VARCHAR2(30)

Tamanho variável. Ocupa apenas o necessário. Use por padrão.

DATE

Inclui hora. Use TO_CHAR para formatar a saída.

BOOLEAN

TRUE, FALSE e NULL. Exclusivo do PL/SQL. Não existe como coluna SQL.

Atributo %TYPE

O atributo %TYPE ancora o tipo de uma variável ao tipo exato de uma coluna ou de outra variável. Se a coluna mudar, a variável se adapta automaticamente na próxima compilação.

```
DECLARE
  -- Ancora o tipo da coluna ename
  v_nome   emp.ename%TYPE;
  v_salario emp.sal%TYPE;
  v_cargo  emp.job%TYPE;
  -- Ancora em outra variavel
  v_backup v_salario%TYPE;

BEGIN
  SELECT ename, sal, job
  INTO   v_nome, v_salario, v_cargo
  FROM   emp
  WHERE  empno = 7839;

  DBMS_OUTPUT.PUT_LINE(v_nome);
END;
/
```

O que é %TYPE

Ancora a variável ao tipo exato da coluna da tabela. Não é preciso saber o tipo: o compilador resolve automaticamente.

Manutenção automática

Se o tipo da coluna mudar no banco, a variável se adapta automaticamente na próxima compilação. Zero impacto no código.

Dica de Produção

Use %TYPE em produção para evitar incompatibilidade de tipo no SELECT INTO. Execute: A02_03_atributo_type.sql

Escopo e Visibilidade

Uma variavel declarada em um bloco e visivel nesse e em todos os blocos internos. O contrario nao e verdadeiro. Execute: A02_05_escopo_blocos.sql

```
<<EXTERNO>>
DECLARE
  v_escopo VARCHAR2(20) := 'externo';
  v_contador NUMBER := 10;
BEGIN

  DECLARE -- bloco interno
    v_interno VARCHAR2(20) := 'interno';
    v_contador NUMBER := 99; -- shadowing
  BEGIN
    -- v_escopo = 'externo' (visivel)
    -- v_contador = 99 (local)
    -- <<EXTERNO>>.v_contador = 10
    NULL;
  END;

END;
/
```

Interno enxerga Externo

Bloco interno acessa variaveis do bloco externo diretamente. v_escopo declarada no externo e visivel dentro do DECLARE/BEGIN interno.

Externo NAO enxerga Interno

Bloco externo nao acessa variaveis declaradas no interno. Tentar usar v_interno fora do sub-bloco gera PLS-00201: identifier must be declared.

Shadowing: nome duplicado

Quando interno redeclara o mesmo nome (v_contador := 99), o bloco interno usa a variavel local. A externa fica oculta mas nao e destruida.

Label <<NOME>>

Qualifica a variavel externa com shadowing: <<EXTERNO>>.v_contador acessa o valor 10 mesmo dentro do bloco interno onde v_contador = 99.

Declaração de Variáveis e Constantes

A seção DECLARE precede o BEGIN e contém todas as variáveis e constantes do bloco.

```
identificador [CONSTANT] tipo [NOT NULL] [:= | DEFAULT expressão];
```

v_nome VARCHAR2(50)

v_sal NUMBER(10,2) := 0; -- inicializado

c_taxa CONSTANT NUMBER := 0.15

v_ok BOOLEAN NOT NULL := TRUE; -- obriga valor

Sem valor inicial → NULL

CONSTANT exige inicialização. Reatribuição gera PLS-00363.

NOT NULL exige valor sempre

DEFAULT e := são equivalentes, use DEFAULT para clareza semântica.

Script: A02_04_not_null_constant.sql

Demonstra CONSTANT, NOT NULL e DEFAULT em ação.

Scripts: A02_01 e A02_02

Execute A02_01 (declaração básica) e A02_02 (tipos escalares) antes de prosseguir.

Pegadinha: NULL

Variável sem valor inicial é NULL, não zero. BOOLEAN NULL não é FALSE. Sempre inicie.

Dica de Produção

Declare sempre com tamanho: VARCHAR2(200), NUMBER(10,2). Use %TYPE para herdar o tipo da coluna.

Operadores em PL/SQL

Atribuição, comparação, lógicos e concatenação

PL/SQL herda os operadores SQL e adiciona := para atribuição. Atenção: := atribui, = compara.

:= Atribuição

Atribui valor a variável. Ex.: v_sal := 1500;

= Igualdade

Compara dois valores. IF v_sal = 1500 THEN

<> != ~= ^= Diferente

Quatro formas equivalentes de desigualdade.

< > <= >= Relacionais

Ordem em NUMBER, DATE e VARCHAR2.

AND OR NOT Lógicos

Combinam booleanos. NOT NULL retorna NULL.

|| Concatenação

Junta strings. 'Olá ' || v_nome.

+ - * / ** Aritméticos

Inclui exponenciação (**) e divisão real.

BETWEEN IN LIKE IS NULL

Faixa, lista, padrão e teste de nulo.

Funções SQL no PL/SQL

Quase todas as funções SQL podem ser usadas em PL/SQL

Funções single-row funcionam direto. Agregação só dentro de SQL embutido.

Numéricas

ROUND, TRUNC, MOD, POWER, ABS, SIGN, CEIL, FLOOR.

Datas

SYSDATE, ADD_MONTHS, MONTHS_BETWEEN, LAST_DAY, NEXT_DAY, TRUNC, ROUND.

Gerais / NULL

NVL, NVL2, COALESCE, NULLIF, DECODE, CASE.

Agregação (só em SQL)

SUM, AVG, COUNT, MIN, MAX — usar em SELECT INTO.

Caracteres

UPPER, LOWER, INITCAP, LENGTH, SUBSTR, INSTR, LPAD, RPAD, TRIM, REPLACE.

Conversão

TO_CHAR, TO_NUMBER, TO_DATE. Sempre use máscara explícita.

Pseudocolunas

USER, SYSDATE, ROWNUM, ROWID, SEQ.NEXTVAL, SEQ.CURRVAL.

Pegadinha

Funções de agregação fora de SQL geram PLS-00204.

SELECT INTO

Como ler dados do banco para variáveis PL/SQL

Sintaxe: `SELECT col1, col2 INTO v1, v2 FROM tabela WHERE ...` Deve retornar EXATAMENTE 1 linha.

Regra de ouro

`SELECT INTO` precisa retornar uma única linha. Nem mais, nem menos.

Ordem importa

Colunas e variáveis combinam por posição. Use `%TYPE` para tipos.

NO_DATA_FOUND

Disparada quando a query não retorna nenhuma linha.

TOO_MANY_ROWS

Disparada quando a query retorna 2 ou mais linhas.

Funções de agregação

`SUM`/`AVG`/`COUNT`/`MAX`/`MIN` não levantam `NO_DATA_FOUND` — retornam `NULL`.

Use WHERE PK

Sempre filtre por chave primária ou única para garantir 1 linha.

Tratamento

Capture as duas exceções no bloco `EXCEPTION`.

Pegadinha

`SELECT COUNT(*) INTO v FROM ...` nunca dá `NO_DATA_FOUND`.

Hora de Praticar!

Abra o SQL Developer ou SQL*Plus e execute cada script abaixo.

```
SET SERVEROUTPUT ON      -- execute isso antes de qualquer script
```

```
A02_01_declaracao_basica.sql
```

```
Declaração de variáveis, := e DEFAULT
```

```
A02_02_tipos_escalares.sql
```

```
NUMBER, VARCHAR2, CHAR, DATE e BOOLEAN
```

```
A02_03_atributo_type.sql
```

```
%TYPE com colunas da tabela EMP - requer schema SCOTT
```

```
A02_04_not_null_constant.sql
```

```
NOT NULL, DEFAULT e CONSTANT
```

```
A02_05_escopo_blocos.sql
```

```
Escopo e visibilidade em blocos aninhados
```

```
A02_06_operadores.sql
```

```
Todos os operadores PL/SQL
```

```
A02_07_funcoes_sql.sql
```

```
Funções de texto, número, data e conversão
```

```
A02_08_exercicio_modelo.sql
```

```
Exercício: SELECT INTO + %TYPE + BLAKE
```

```
A02_09_sequencias_transacoes.sql
```

```
NEXTVAL, DML, COMMIT, ROLLBACK, SAVEPOINT
```

Revisão do Módulo M02

Identificadores: letra inicial, max 30 chars, case-insensitive. Aspas duplas tornam case-sensitive.

Literais: numérico (inteiro, real, científico), caracter, string e booleano (TRUE/FALSE/NULL).

Tipos escalares: BINARY_INTEGER, NUMBER(p,s), CHAR(n), VARCHAR2(n), DATE, BOOLEAN.

Atributo %TYPE: ancora variável ao tipo de coluna ou outra variável. Essencial em produção.

Escopo: bloco interno enxerga o externo. Use labels (<<NOME>>) para qualificar nomes duplicados.

Declarações: CONSTANT exige init e não muda. NOT NULL obriga valor. DEFAULT equivale a :=.

Próxima aula: M03 - Estruturas de Controle

IF/CASE, LOOP, WHILE, FOR e controle de fluxo